Informe y análisis del desafío #2: UdeABook.

Pablo Colorado Correa

1. Introducción

En este desafío, abordare la temática y el modelo de Programación Orientada a Objetos (POO), cuyo propósito principal es el de desarrollar desde cero la capa lógica de una red social denominada **UdeABook**, enfocada en gestionar eficientemente la información de usuarios, álbumes, fotos, grupos y relaciones de amistad.

Este informe detalla el análisis que decidí aplicar para resolver la problemática planteada, describiendo la estructuración de archivos de almacenamiento permanente, las clases fundamentales con sus respectivos atributos y métodos, y los criterios de eficiencia.

2. Descripción del problema

El reto consiste en implementar en C++ un sistema que simule las funcionalidades básicas de una red social interactuando a través de la consola. De cada usuario en la plataforma se requiere gestionar:

- Su nombre de usuario único, fecha de nacimiento, correo electrónico, ruta de foto de perfil, ciudad, país y fecha de inscripción.
- Cero o más amigos, un conjunto de mensajes enviados y recibidos, solicitudes de amistad y sugerencias de amistad rankeadas según los amigos en común.
- Cero o más álbumes con configuraciones de privacidad (públicos, privados o de amigos), los cuales contienen un conjunto de fotos con fecha de captura, tamaño, ruta de almacenamiento y conteo de likes.

Asimismo, el sistema debe soportar funcionalidades esenciales como la carga y actualización de datos desde archivos persistentes, inicio de sesión, administración de amistades, fotos y likes, visualización de sugerencias de amistad, administración de mensajes con restricciones de longitud y búsqueda por rangos de fecha, y una medición del consumo de recursos (memoria e iteraciones) tras cada ejecución.

3. Análisis utilizado

Para dar solución al desafío, estructuraré el diseño modular y el manejo de almacenamiento permanente mediante archivos de texto plano (.txt):

- **Usuarios:** Almacena la información personal y credenciales de cada miembro registrado en la plataforma.

- **Amistades y Solicitudes:** Registran los vínculos establecidos y las peticiones pendientes entre usuarios.
- **Álbumes y Fotos:** Contienen rutas absolutas en notación Linux (incluyendo la foto de perfil y las imágenes organizadas por álbumes).
- **Mensajes:** Gestionan la comunicación textual entre usuarios con un límite de hasta 512 caracteres y estados de lectura.

Para este desafío, decidí implementar las siguientes clases primordiales:

- **Plataforma:** Funciona como el núcleo central del sistema y gestiona la carga y guardado de datos desde el almacenamiento permanente, además de coordinar la interacción global y las métricas de consumo.
- **Usuario:** Modela el perfil individual, conteniendo sus datos personales, colecciones dinámicas de amigos, mensajes, solicitudes y álbumes.
- **Álbum:** Agrupa las fotografías pertenecientes a una categoría específica, administrando su nivel de privacidad.
- **Foto:** Representa cada imagen con sus metadatos de tamaño, fecha, likes y ruta de acceso.
- **Mensaje:** Estructura los textos intercambiados entre usuarios, controlando el emisor, receptor, fecha y estado de lectura.
- **SolicitudAmistad, SugerenciaAmistad y Amigos:** Clases auxiliares orientadas a el manejo de las solicitudes de amistad, amistades aceptadas por el usuario y con esto las sugerencias

Para el desarrollo en C++ bajo la POO, hare uso de librerías estándar esenciales como <iostream>, <fstream>, <sstream>, <string>, <cstdlib>, <ctime>, <random> y <chrono>, evitando el uso de contenedores de la STL para las estructuras principales del núcleo con el fin de garantizar el uso obligatorio de memoria dinámica y optimizar la eficiencia exigida.

4. Para Finalizar

En este desafío se adoptará eficiencia algorítmica y el control del uso de memoria, evitando copias innecesarias mediante el uso adecuado de referencias y estructuras de datos dinámicas construidas desde cero.

Para concluir, la aplicación de la Programación Orientada a Objetos mediante módulos independientes (.h y .cpp) permite estructurar el programa de forma clara,

para un diseño escalable, mantenible y preparado para cumplir estrictamente con todas las restricciones formales y de rendimiento evaluadas en el curso.

